# CDEF step by step

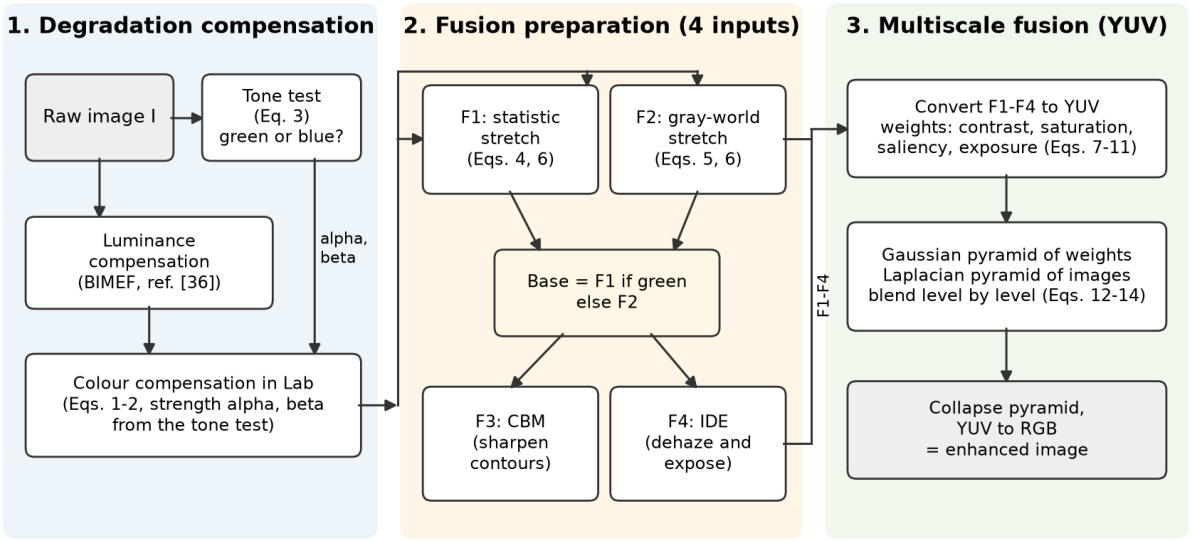
What every step of the method does to an image, and why it has the effect it has. Every number and picture in this document comes from running our implementation (*cdef/*) on real images.

## How to read this

Each step has the same parts: **what it does** (the operation), **what it does to the example images** (measured), and a shaded box, **why this happens**, which explains the mechanism behind what you see. Three UIEB images are followed through the whole method:

| Image | What is wrong with it | Used to show |
|---|---|---|
| 18_img_ (seabed) | Strong green cast, red almost gone (mean R = 0.05) | Colour compensation, tone maps, CBM |
| 518_img_ (diver) | Strong cyan cast, haze in the distance | IDE, the fusion weights |
| 12290 (sharks at night) | Very dark (mean 0.10) | Luminance compensation (BIMEF) |

Pixel values are on a 0 to 1 scale throughout (0 = black, 1 = full brightness). Section 1 explains the few image-processing ideas the later sections rely on; skip it if they are familiar.




The pipeline. Phase 1 repairs the image, phase 2 makes four enhanced versions, phase 3 fuses them.

# 1. Background you need

## Colour spaces

An image can be described in different coordinates. CDEF uses three, each for what it makes easy:

| Space | Channels | Why CDEF uses it |
|---|---|---|
| RGB | Red, green, blue intensity | How images are stored. Brightness and colour are mixed in every channel. |
| CIELab | L* lightness (0 to 100); a* green (-) to red (+); b* blue (-) to yellow (+) | Colour and lightness are separate, and a* and b* are opponent axes: a grey pixel has a* = b* = 0. A colour cast is simply a* and b* being shifted away from 0. |
| YUV | Y luma (brightness); U, V colour differences (about 0 for grey) | Also separates brightness from colour, but is a linear (matrix) transform of RGB, so it is cheap and exact to invert. Used for the fusion. |

## Filters and morphology

- **Gaussian blur** replaces each pixel by a weighted average of its neighbours; sigma sets the radius. A box filter is the same with equal weights.
- **Laplacian** is a 3 x 3 filter that responds to edges and fine detail and is zero on flat areas.
- **Erosion** replaces a pixel by the minimum over a neighbourhood (the structuring element), **dilation** by the maximum. An **opening** (erode then dilate) removes bright features smaller than the element; a **closing** (dilate then erode) removes dark ones. A **top-hat** is the difference between the image and its opening or closing: exactly the small features that were removed.

## Histograms and stretching

A channel's histogram counts how many pixels have each value. A low-contrast image has a narrow histogram. A **linear stretch** maps a chosen low value to 0 and a chosen high value to 1, widening the histogram; values outside the limits are clipped to 0 or 1.

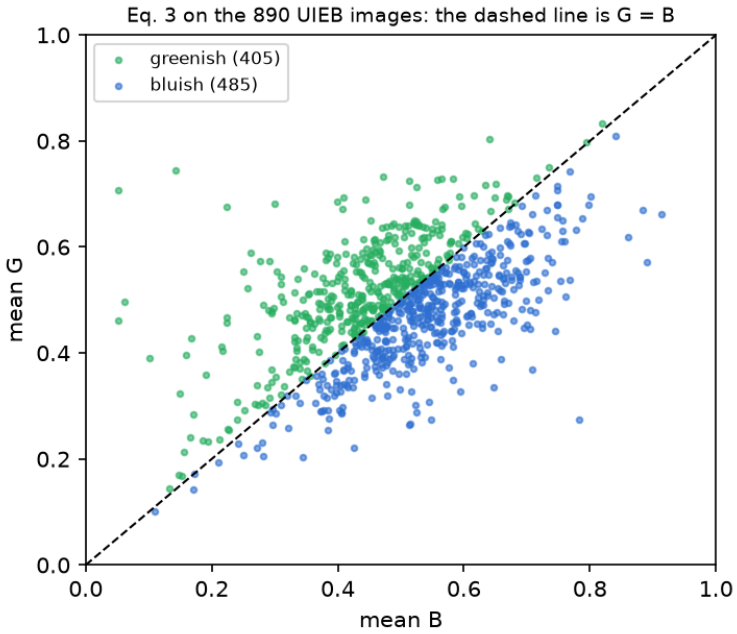
## Pyramids

A **Gaussian pyramid** repeatedly blurs and halves an image, giving coarser and coarser copies. A **Laplacian pyramid** stores, at each level, the detail lost between one Gaussian level and the next, plus the coarsest copy. Adding the levels back up rebuilds the image exactly. Level 1 holds the finest detail (edges, texture); the last level holds only the overall brightness and colour.

## 2. Step 1: tone test (Eq. 3)

**What it does.** Compares the average green and blue values of the raw image. If mean G > mean B the image is *greenish* and gets alpha = 0.5, beta = 0.15; otherwise it is *bluish* and gets alpha = 1.0, beta = 0.3. The decision is used twice: for the strength of colour compensation (step 3) and to choose which tone map is sharpened and dehazed (steps 5 and 6).

**On the data.** Of the 890 UIEB images, 405 are greenish and 485 bluish. The seabed image (mean G 0.69, B 0.41) is greenish; the diver (G 0.70, B 0.80) and the night sharks (G 0.101, B 0.110) are bluish.




Every UIEB image by its mean G and mean B. The dashed line is the decision boundary.

**Why this happens**

Water absorbs red first, then green, then blue, so underwater casts fall into two families. Green light survives better than blue in many coastal waters, so a greenish image has lost less and needs a gentler correction; a bluish one gets twice the strength.

alpha (for a*, the red-green axis) is larger than beta (for b*, the yellow-blue axis) because red is the colour that is lost most.

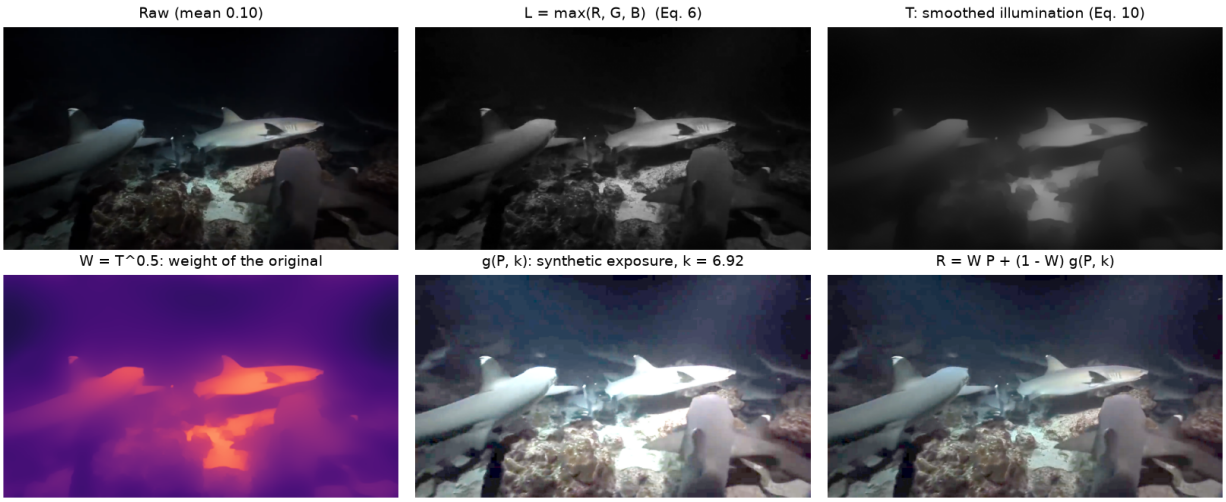
Note the night-shark image: it has almost no cast at all (G and B differ by 0.009), yet the binary test still calls it bluish and applies the strong setting. Images near the line are the method's weak spot.

# 3. Step 2: luminance compensation (BIMEF)

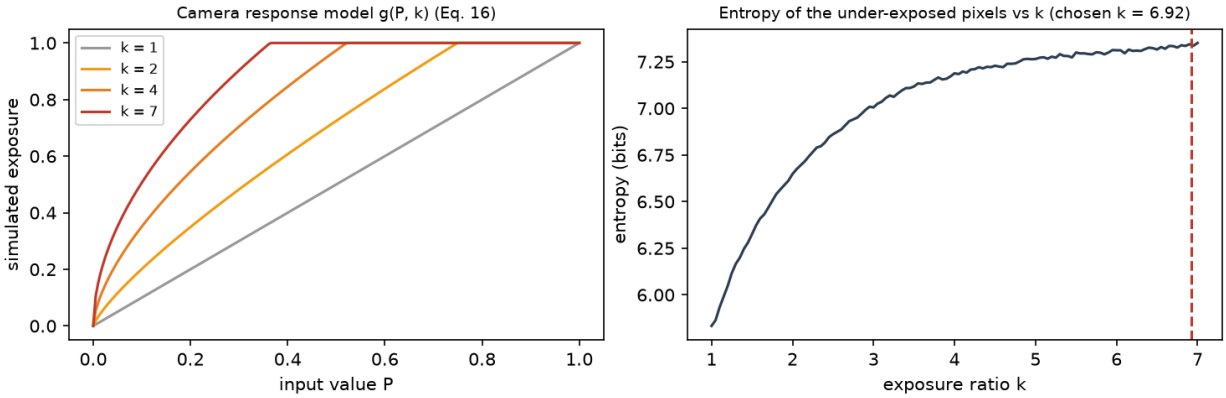
**What it does.** Brightens dark regions and leaves well-lit ones alone. It is a small exposure fusion in itself, in four sub-steps:

- **Initial illumination L = max(R, G, B)** at every pixel (Eq. 6).
- **Smoothed illumination T**: L is smoothed while keeping strong edges, by solving a large sparse linear system (Eq. 10) on a half-size copy. T says how well lit each pixel is.
- **Synthetic exposure g(P, k)**: a brighter copy of the image made with a camera response model, as if the photo had been taken with k times more exposure (Eq. 16).
- **Blend**: R = W P + (1 - W) g(P, k) with $W = T^{0.5}$. Well-lit pixels (T near 1) keep the original; dark pixels (T near 0) take the brightened copy.

The exposure ratio k is chosen automatically: among the pixels with T < 0.5, it is the k in [1, 7] that maximises the entropy of their brightness after the transform (Eqs. 17-21), searched on a 50 x 50 copy.




BIMEF on the night-shark image: 98% of pixels have T < 0.5, k = 6.92, and the mean brightness goes from 0.10 to 0.31.




Left: the camera response model for several k. Right: how k is chosen for the night-shark image.

**On the data.**

| Image | Share of pixels with T < 0.5 | k | Mean brightness before → after |
|---|---|---|---|
| Night sharks (12290) | 98% | 6.92 | 0.10 → 0.31 |
| Diver (518_img_) | under 0.5% | 6.56 | 0.51 → 0.57 (only the diver's dark suit changes much) |
| Seabed (18_img_) | 0% | 1.00 | unchanged: nothing is under-exposed |

**Why this happens**

**Why max(R, G, B).** In the Retinex model a pixel is reflectance x illumination, with reflectance at most 1 in every channel. So the illumination is at least as bright as the brightest channel; the max is the simplest estimate.

**Why the smoothing keeps edges.** The weight of each pixel's smoothness term is 1 / (|gradient summed along a 5-pixel line| x |gradient| + eps). Inside fine texture the gradients point both ways and cancel in the sum, so the weight is large and texture is smoothed away. At a real boundary between a lit and an unlit region the gradients add up, the weight is small and the edge is kept. That is why T in the figure is blurry inside each region but sharp around the sharks: brightening does not bleed across object edges.

**Why entropy picks k.** In a dark image the pixel values are crowded into a few dark levels, which carries little visible information. Brightening spreads them over more levels, which raises the entropy. For a very dark image the entropy keeps rising up to the upper limit (right plot), so k ends near 7: the range limit matters for such images.

**Why the diver image changes so little despite k = 6.56.** k is chosen only from the few dark pixels (the suit), but for every other pixel T is near 1, so W is near 1 and the original is kept. The blend applies the brightening only where it was measured to be needed.

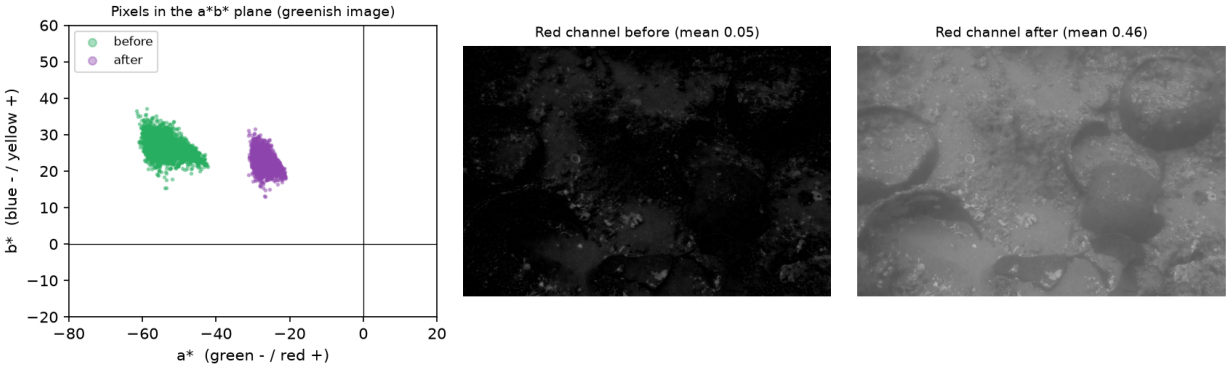
**Why the seabed image is unchanged.** No pixel has T < 0.5, so there is nothing to compensate and k = 1.

# 4. Step 3: colour compensation in Lab (Eqs. 1-2)

**What it does.** Converts the image to CIELab and removes part of the colour cast from the two colour channels, then converts back:

$$a^{*\prime} = a^* - \text{alpha} \cdot M \cdot G(a^*) \qquad b^{*\prime} = b^* - \text{beta} \cdot M \cdot G(b^*)$$

G is a Gaussian blur (sigma = 1 pixel) and M is a mask that is 0 on very bright pixels (mean above 0.85, light sources) and 1 elsewhere. L* is not touched.




Seabed image. Left: a sample of pixels in the a*b* plane before and after. Middle and right: the red channel before and after.

**On the data.**

| Image | alpha, beta | mean a* before → after | mean b* before → after | mean L* |
|---|---|---|---|---|
| Seabed (greenish) | 0.5, 0.15 | -54.5 → -27.3 | 26.8 → 22.8 | 63.7 → 63.7 |
| Diver (bluish) | 1.0, 0.3 | -30.2 → 0.0 | -24.8 → -17.3 | unchanged |

**Why this happens**

**Why the numbers are exactly (1 - alpha) and (1 - beta) times the old ones.** With sigma = 1 pixel the blur barely changes a smooth channel, so G(a*) is almost a*, and M = 1 (these images have no light sources). Eq. 1 then reduces to a*' = (1 - alpha) a*: the greenish setting halves a* (-54.5 to -27.3) and keeps 85% of b*; the bluish setting removes a* completely and keeps 70% of b*. The whole cloud of pixels slides towards the grey point (0, 0) in the a*b* plot, keeping its shape.

**Why red appears from nowhere.** Moving a* from strongly negative (green) towards 0 at the same lightness means, back in RGB, less green and more red. The new red values are computed from L* and the new a*, and L* carries all the scene's structure. So the red channel, nearly black and empty before (mean 0.05), now holds a full copy of the image's texture (mean 0.46). This is why the compensation is done before any stretching: stretching the original red channel would only magnify noise.

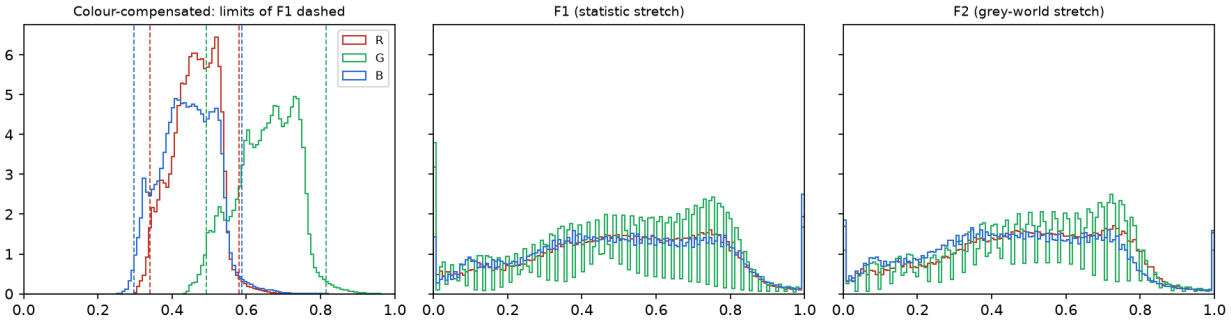
**Why it does not remove the whole cast.** The seabed image still looks pale green after this step (a* = -27). Removing the cast completely (alpha = beta = 1) gives a nearly grey image (paper Fig. 4), because a* and b* would be pushed to 0 everywhere, not just on average. The rest of the cast is removed by the tone maps in the next step.

**Why the mask.** Light sources are the one place where the true colour is known (the lamp's colour). Compensating them would tint them; the mask leaves them alone.

# 5. Step 4: tone maps F1 and F2 (Eqs. 4-6)

**What it does.** Stretches each colour channel *separately* so that a low limit maps to 0 and a high limit maps to 1 (Eq. 6), clipping values outside. The two inputs differ only in how the limits are chosen:

- **F1, statistic (Eq. 4):** limits at the channel's mean - 2 x standard deviation and mean + 2 x standard deviation.
- **F2, grey-world (Eq. 5):** limits at quantiles: the level b x (grey mean / channel mean) from the bottom and from the top, b = 0.01. A channel darker than the overall grey gets a larger level, so more of it is clipped and it is stretched harder.




Seabed image, per-channel histograms. Left: after colour compensation, with the F1 limits dashed. Middle: F1. Right: F2.

**On the data.** F1 limits on the seabed image: red 0.34 to 0.58, green 0.49 to 0.81, blue 0.30 to 0.59. Channel means after colour compensation [0.46, 0.654, 0.443], after F1 [0.499, 0.5, 0.497], after F2 [0.498, 0.504, 0.459].

**Why this happens**

**Why the cast disappears.** Green's histogram sits higher than red's and blue's (left plot), so its limits are higher too (0.49 to 0.82 against 0.34 to 0.58 for red). Each channel's own mean lands in the middle of its own range, so after F1 every channel has a mean of almost exactly 0.5 and the same spread. Equal channels are grey: the remaining cast is gone. A per-channel stretch is a white balance and a contrast boost in one.

**Why F1 always comes out mid-grey.** mean - 2 std maps to 0 and mean + 2 std maps to 1, so the mean always maps to exactly 0.5. Even the very dark night-shark image comes out of F1 with mean 0.49.

**Why F2 is usually brighter or keeps the image's own brightness.** Its limits are near the 1% and 99% points of the data, which for a skewed histogram are not symmetric around the mean. On the diver image F2 keeps L* = 73 while F1 gives 55; on the dark sharks F2 stays dark (0.31). That is why the fusion has both: F1 and F2 offer two different exposures.

**Why the green histogram turns into a comb in F1.** Green occupied only about a third of the range, so it had roughly a third as many distinct levels. Stretching spreads the same levels over the whole range and leaves gaps between them. Visually this is invisible, but it shows that stretching cannot create detail, only spread what is there.

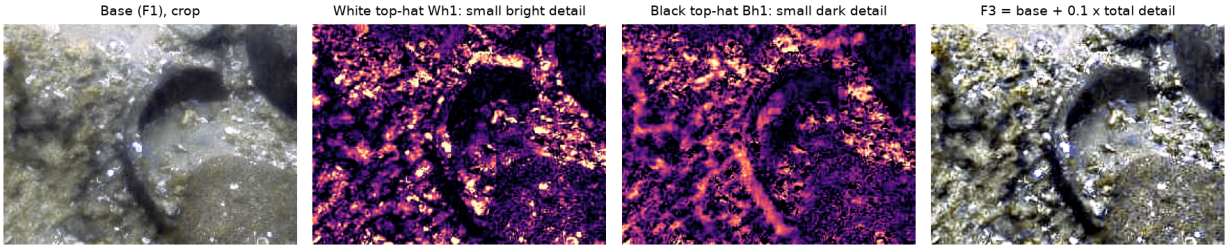
**Why about 5% of each channel is clipped in F1.** For a bell-shaped histogram about 2.5% of the values lie beyond each of mean +/- 2 std. Those become pure black or white, which adds punch.

**Why the tone decides the base for steps 5-6.** The paper averaged 4880 images into a mean greenish and a mean bluish image and found F1 suits the greenish one and F2 the bluish one. F3 and F4 are built from that base, so they inherit its neutral colour.

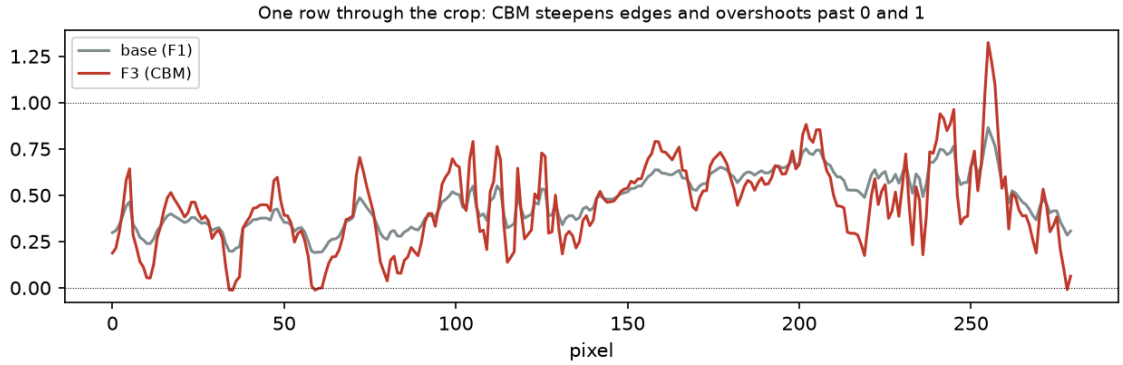
## 6. Step 5: F3, Contour Bougie Morphology (CBM)

**What it does.** Sharpens contours with morphology. Using hollow square structuring elements (only the outer ring of a 15 x 15 and a 5 x 5 square), it computes the white top-hat Wh (small bright details) and the black top-hat Bh (small dark details), repeated over 10 iterations, and adds them back:

$$F3 = \text{base} + 0.1 \cdot (\ \text{sum of } (Wh_i - Bh_i) + \text{sum of differences between iterations}\ )$$




Seabed image, crop. The white top-hat finds bright specks; the black top-hat finds dark gaps and outlines.




One row of pixels through the crop: CBM makes bright details brighter and dark ones darker, and overshoots past 0 and 1 at strong edges.

**Why this happens**

**Why it sharpens.** Adding Wh makes small bright features brighter; subtracting Bh makes small dark ones darker. Large smooth areas are unchanged, because the opening and closing leave them intact. The result is more local contrast exactly at small features and outlines (the profile plot).

**Why the 10 iterations add up to one.** The open-close filter does not change its own output: applied a second time it changes nothing. We checked this on every example image. So all 10 top-hats are identical and the differences between iterations are 0, and the formula collapses to 0.1 x 10 x (Wh - Bh), i.e. **F3 = base + Wh - Bh**, the classic top-hat contrast enhancement.

**Why a ring-shaped element.** A filled square removes any feature smaller than itself. A ring only 'fits' around features that are enclosed, so the top-hats respond to outlines and contours of objects rather than to every textured patch.

**Why values leave [0, 1].** A bright detail next to 1 gets brighter still. 11% of the F3 values on the seabed image fall outside [0, 1]. They are not clipped here, as in the authors' code: the fusion then sees the full edge strength, and the final output is clipped after fusion.

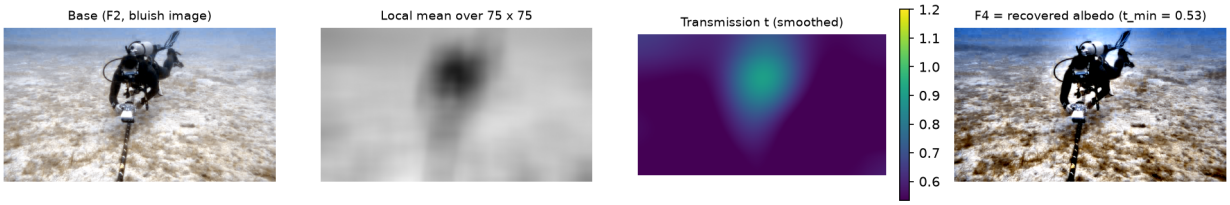
# 7. Step 6: F4, Image Dehazing and Exposure (IDE)

**What it does.** Removes haze with a physical model. The classic haze model says every observed pixel is a mix of the true scene J and the ambient water light A:

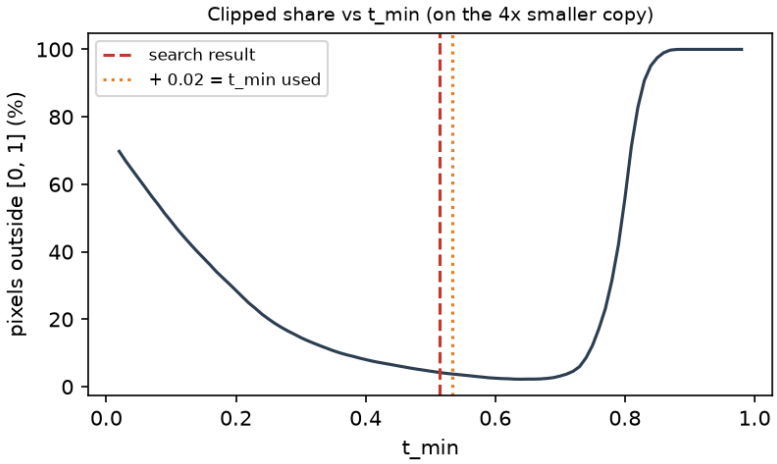
$$I = J \cdot t + A \cdot (1 - t)$$

t is the transmission: 1 means a clear view, near 0 means the pixel shows mostly water. IDE adds an exposure term so that the dehazed result is not too dark, then:

- **Ambient light A**, per channel: the brightest value of a 10 x 10 local minimum (a robust 'colour of the water').
- **Transmission t** in closed form (Eqs. 12-17), assuming each 75 x 75 patch has an average true brightness of one half; then smoothed by two passes of a 101 x 101 mean.
- **Strength $t_{min}$**, the one free parameter, found by search (Eq. 22), see below.
- **Recovery**: invert the model, scene = gain x (I - A(1 - t)) / (t A) (Eq. 20).




Diver image (base F2). Bright, uniform sand gets low t (strong dehazing); the dark diver gets high t (little change).




How strong to dehaze: the share of recovered pixels outside [0, 1] for each t_min. The search finds the lowest point (2.23%), then moves left until 2 points higher, then adds 0.02: t_min = 0.534.

**Why this happens**

**Why bright areas get low t.** Each patch is assumed to have average true brightness one half. A patch that is brighter than that can only be explained by haze adding light, so the brighter (and closer to A) a patch is, the lower its t. The diver's dark suit is darker than the assumption, so its t is high and it is left mostly alone. This is right for haze, but it also means genuinely bright objects are mistaken for haze (see the halo below).

**Why dividing by t removes haze.** Subtracting A(1 - t) removes the water light that was added; dividing by t restores the contrast the haze reduced. With t around 0.53 to 0.79 here, local contrast in the background is roughly doubled, which is why the far sand gets its texture back.

**Why the clipped-share curve is U-shaped.** A small t_min means very strong dehazing: many pixels are pushed below 0 or above 1 (left side). A t_min near 1 makes ln t_min near 0, and the closed-form t becomes unstable, so recovery fails again (right side). The search takes a point on the left slope, slightly stronger than the minimum, which trades a few clipped pixels for more dehazing.

**Why per-channel A also corrects colour.** Each channel is divided by its own A, so the water colour maps to the same value in every channel: a second white balance. On the diver image A = (1, 1, 1) because the base is already balanced, so the colour barely changes. On a dark greenish image (UIEB 332_img_) A for blue is only 0.79, so blue is boosted and F4 comes out bluish (b* = -27); the fusion then tempers it.




C60 12753: a bright ROV in dark water.

**Why this happens**

**Why a ring forms around bright objects.** The bright ROV gets a low t (mistaken for haze, above). The 101 x 101 box smoothing spreads that low t into the dark water around it. There, dividing dark water by a small t pushes it darker still, which draws a dark ring around the ROV. The fusion's squared weights then favour F4's strong edge there, so the ring survives into the output. The authors' code shows the same ring.

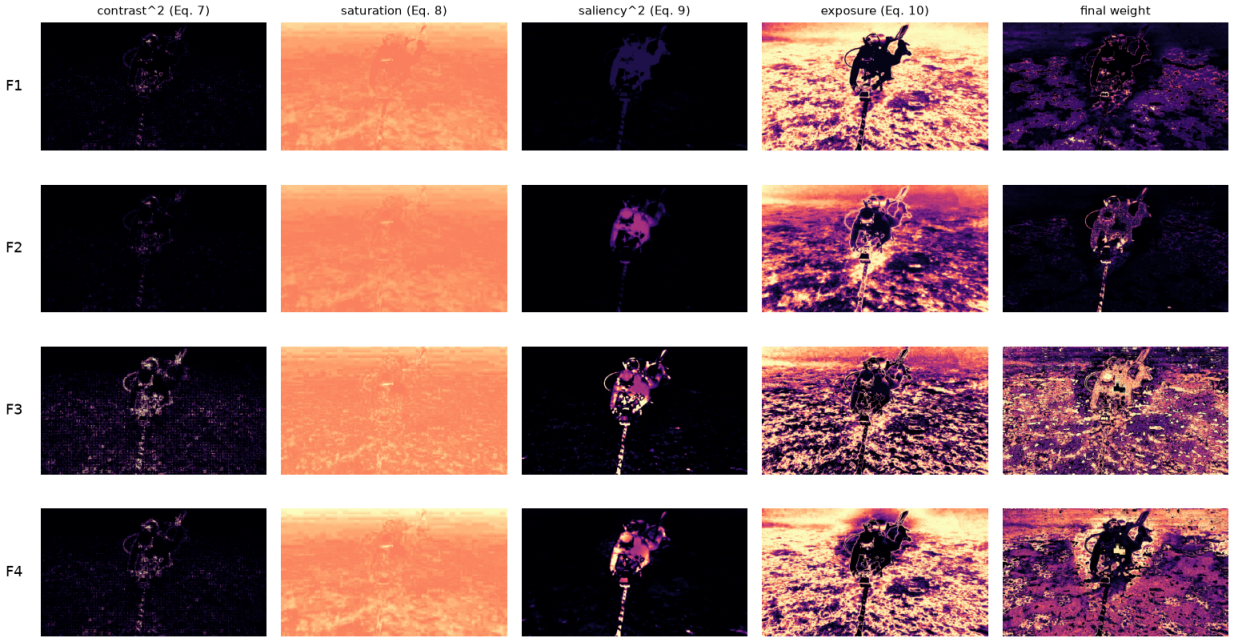
**Why the flat water shows contours.** The water is nearly uniform, so its pixels differ by only one or two 8-bit levels. Dehazing multiplies those differences, turning invisible steps into visible bands.

# 8. Step 7: multiscale fusion in YUV (Eqs. 7-14)

**What it does.** Converts F1 to F4 to YUV and gives every pixel of every input a weight, the product of four measures:

| Measure | Formula | High where | Exponent |
|---|---|---|---|
| Contrast (Eq. 7) | \|Laplacian of Y\| | Edges and fine texture | 2 |
| Saturation (Eq. 8) | \|U\| + \|V\| + 1 | Colourful pixels | 1 |
| Saliency (Eq. 9) | $(Y - \text{mean } Y)^2 + (U - \text{mean } U)^2 + (V - \text{mean } V)^2$, after a small blur | Pixels that stand out from the image average | 2 |
| Exposure (Eq. 10) | $\exp(-(Y - 0.5)^2 / (2 \cdot 0.2^2))$ | Mid-grey pixels; low for very dark or very bright | 1 |

The four products are normalised so that the weights of the four inputs add up to 1 at every pixel (Eq. 11), then each input's map is scaled by its mean brightness squared and normalised again (as in the authors' code). Finally the inputs are blended over image pyramids (Eqs. 12-14) and converted back to RGB.




Diver image: the four measures for each input, and the final weight (bright = trusted). Each column of measures shares one colour scale.

**On the data.**

| Input | mean contrast$^2$ | mean saliency$^2$ | mean Y | mean final weight |
|---|---|---|---|---|
| F1 | 0.0087 | 0.0056 | 0.52 | 0.11 |
| F2 | 0.0053 | 0.0082 | 0.71 | 0.05 |
| F3 | 0.0448 | 0.0269 | 0.70 | 0.43 |
| F4 | 0.0181 | 0.0205 | 0.54 | 0.41 |

**Why this happens**

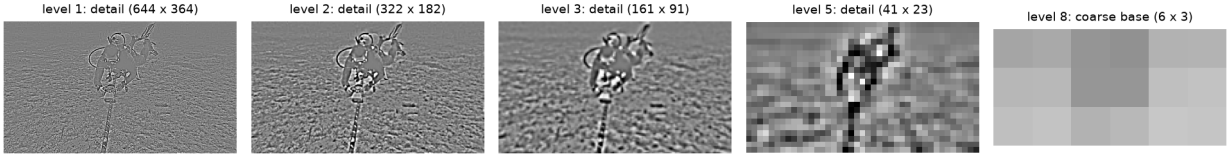
**Why F3 and F4 win most pixels.** Sharpening and dehazing raise local contrast and make features stand out, which is exactly what the contrast and saliency measures reward, and both are squared. F3's mean contrast$^2$ is five times F1's. Together F3 and F4 carry about 84% of the weight on the diver image (and 85% on the seabed).

**Why F4 wins the distant water and F3 the sand.** In the hazy top of the image only the dehazed F4 has texture and a mid-grey exposure. On the near sand F3's added top-hat detail gives the highest contrast.

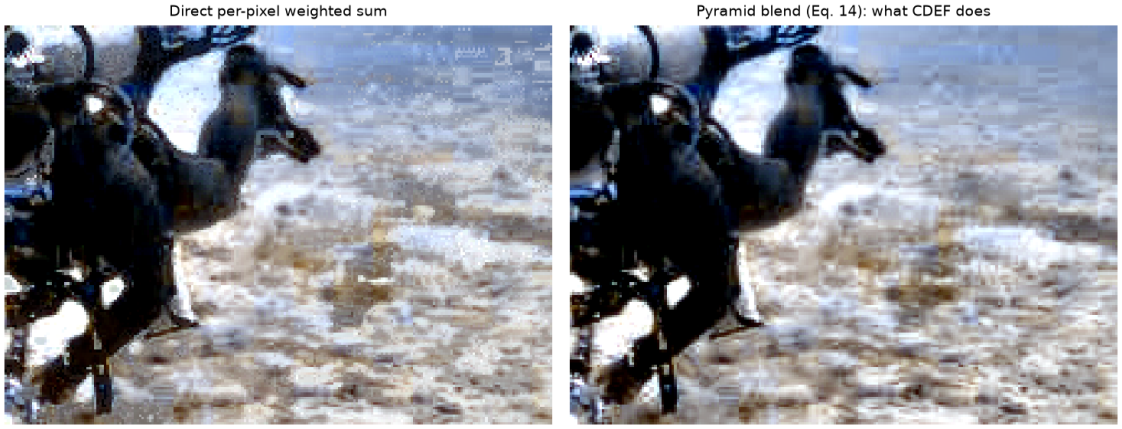
**Why F2 gets the least weight although it is the brightest.** The brightness factor favours F2 (mean Y 0.71 against 0.52 for F1, so about 1.9 times), but F2 is the gentlest stretch: its contrast is the lowest, and at Y = 0.7 its exposure measure is already falling. The product loses.

**Why multiply rather than add the measures.** A product is only high when every measure is high: a sharp but badly over-exposed pixel scores low. Squaring contrast and saliency makes the choice more decisive, which raises sharpness in the result.

## Why blend over a pyramid




Laplacian pyramid of F3's Y channel (8 levels for this 364 x 644 image, Eq. 12). Fine levels hold edges, the last level only the overall brightness.




Diver image, zoomed. Left: the weights applied directly, pixel by pixel. Right: the pyramid blend.

**Why this happens**

**Why a direct weighted sum looks speckled.** The weights can jump from one input to another between neighbouring pixels. Adding the inputs directly copies those jumps into the image: isolated pixels from a brighter or darker input (left image).

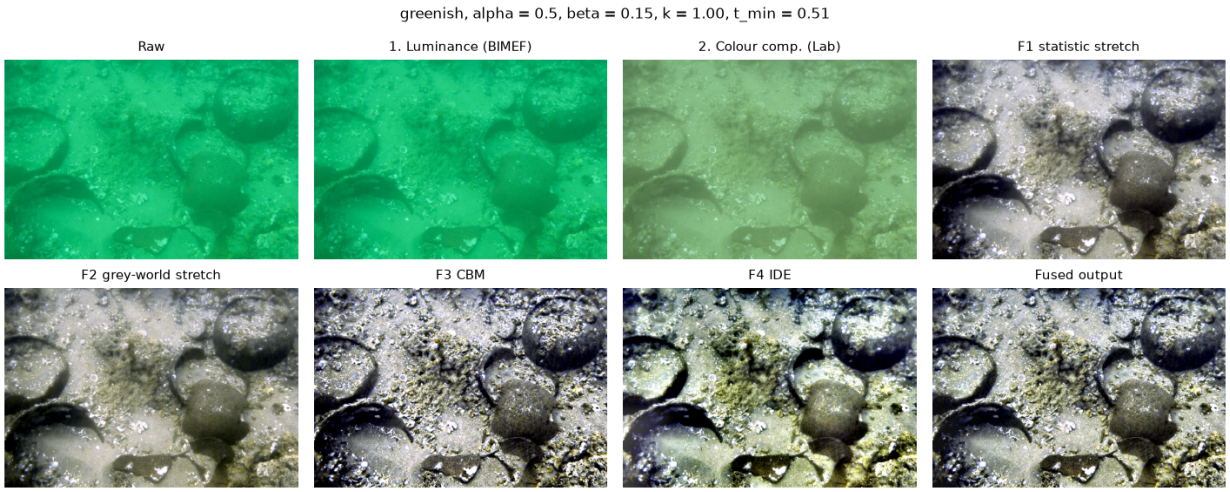
**Why the pyramid fixes it.** At level n, the image detail of that scale is weighted by a correspondingly blurred copy of the weight map (Eq. 14). Fine detail uses sharp, local weights, so edges come from the sharpest input; overall brightness uses very smooth weights, so there are no visible seams where one input takes over from another.

**Why Eq. 12 sets the depth to $\log_2$ of the image size.** Each level halves the image, so after $\log_2(\min(h, w))$ levels the coarsest level is a few pixels across: the blend then works at every scale the image has, for small and large images alike.

**Why YUV.** The contrast and exposure measures need a brightness channel (Y) and the saturation measure needs colour channels (U, V). In RGB every channel mixes both, so a weight chosen for brightness would also move colour.

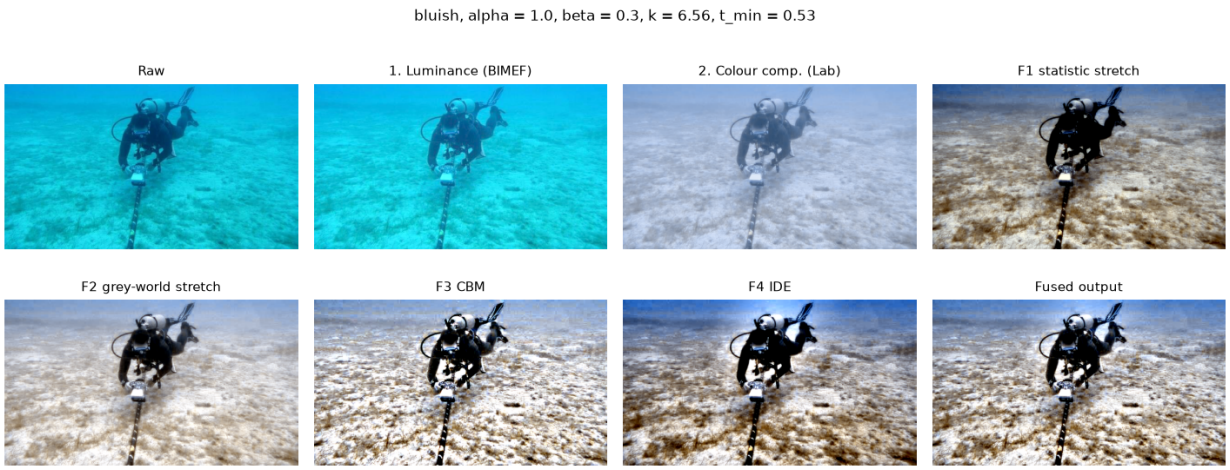
**Why the output is clipped only at the end.** F3 and F4 contain values outside [0, 1] at strong edges. Keeping them through the fusion preserves the full edge strength; one clip at the end keeps the output valid.

# 9. All the steps together



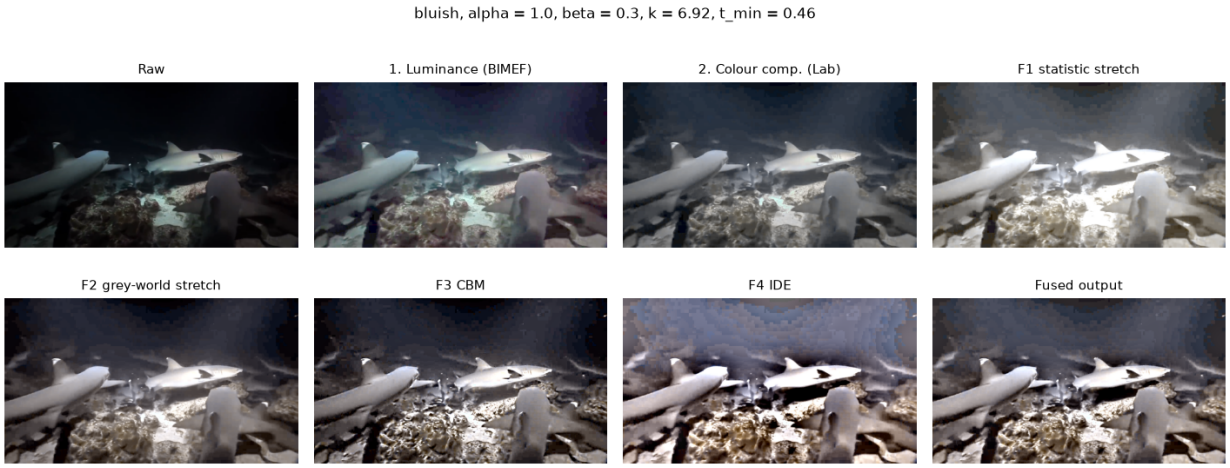

Seabed (greenish).

Greenish: BIMEF does nothing (k = 1); Lab halves the green cast and creates the red channel; F1 removes the rest of the cast; CBM and IDE work from F1; the fusion takes texture mostly from F3 and F4.




Diver (bluish).

Bluish: BIMEF lifts only the diver's suit; Lab removes all of a* and 30% of b*, leaving a pale blue; F1 neutralises it and darkens, F2 neutralises it and keeps it bright; IDE (from F2) removes the haze in the distance.



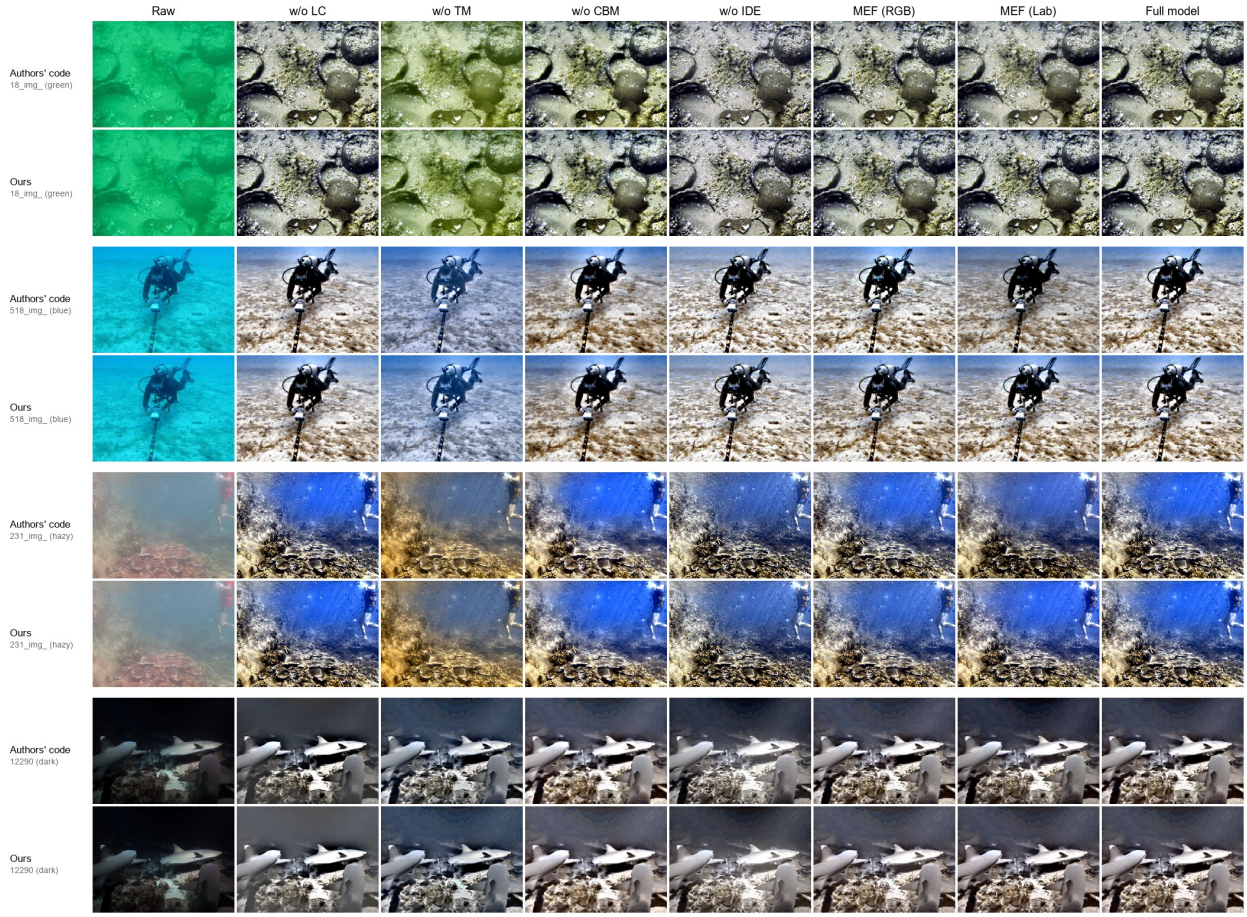

Night sharks (dark, classified bluish).

Dark: BIMEF does the main work (k = 6.92, brightness 0.10 to 0.31); there is hardly any cast to remove; F1 lifts the image to mid-grey while F2 stays dark, giving the fusion two exposures to choose from.

## 10. What the ablation shows, and why

Each ablation removes one step (or fuses in another colour space) and re-scores all 890 UIEB images. The change from the full model, ours (and the paper's):

| Removed | CCF change | What you see | Why |
|---|---|---|---|
| Tone maps (w/o TM) | -25% (-17%) | The cast stays (yellow-green) | The per-channel stretch is what removes the last of the cast (step 4). Without it, the fusion inputs are the pale compensated image, so every input still carries the cast. |
| Luminance (w/o LC) | -9% (-11%) | Darker, flatter in dark scenes | Only matters for images with dark regions: on the seabed image BIMEF already does nothing (k = 1). |
| CBM (w/o CBM) | -2% (-2%) | Slightly softer fine detail | F3 carried about half the weight; without it F4 takes over, which also adds contrast, so little is lost. |
| IDE (w/o IDE) | -3% (-24%) | Distant water a little hazier | F3 takes over and its top-hat contrast partly substitutes. The authors' own code also drops only about 5% here. |
| Fusion in RGB | -13% (-26%) | Small colour shifts | R, G and B are blended with their own weights, so brightness and colour are no longer separated. |
| Fusion in Lab | +0.4% (-17%) | Same as YUV | Lab scaled to [0, 1] has a brightness channel and two opponent colour channels, like YUV, so the fusion behaves the same. The authors' code used unscaled Lab (L up to 100), which breaks the exposure measure. |




Each ablation on four UIEB images, authors' code (top row of each pair) and ours (bottom).

## 11. Summary: each step in one line

| Step | What changes in the image | The mechanism |
|---|---|---|
| 1. Tone test | Nothing directly; picks the strength | mean G vs mean B |
| 2. BIMEF | Dark regions brighter, lit regions unchanged | Edge-preserving illumination map T decides where to blend in a k-times exposure |
| 3. Lab compensation | Cast reduced; red channel filled in | a*, b* scaled by (1 - alpha), (1 - beta) at fixed L* |
| 4. Tone maps F1, F2 | Cast removed, contrast up, two exposures | Each channel stretched on its own range |
| 5. CBM (F3) | Sharper contours, some overshoot | base + white top-hat - black top-hat |
| 6. IDE (F4) | Haze removed in the distance | Invert I = J t + A(1 - t); t from a per-patch assumption |
| 7. Fusion | Best input chosen per pixel, no seams | Contrast, saturation, saliency, exposure weights; pyramids |